

Implementación ambiente de desarrollo
Aplicación Móvil para Fortalecimiento de
Parejas
"AffiniScore"

Tabla de contenido

Datos del documento

Histórico de Revisiones

Versión	Fecha	Descripción/cambio	Autor
1.0	12/06/2026	Inicio documento de implementación de ambiente de desarrollo	Belén Toloza

Información del Proyecto

Organización	Duoc UC. Escuela de Informática y Telecomunicaciones
Sección	002D
Proyecto (Nombre)	AffiniScore
Fecha de Inicio	17/03/2025
Fecha de Término	20/06/2025
Cliente	Cristián Gómez (Academia Tecnológica Triskeledu)
Metodología	ABP / Scrum
Docente	Arturo Vargas Reyes

Integrantes

Rut	Nombre	Correo
21.374.678-2	Belén Toloza	belen.toloza@duocuc.cl
21.571.237-0	Ignacio Álvarez	ignacio.alvarez@duocuc.cl
21.648.445-2	Cristóbal Paredes	cristobal.paredes@duocuc.cl

Introducción

El presente documento tiene como propósito definir el flujo de desarrollo de la aplicación móvil del proyecto **AffiniScore**, estableciendo de manera clara los prerequisites, pasos de instalación y configuración del entorno de trabajo. Esta guía busca asegurar una correcta implementación del entorno de desarrollo, minimizando errores recurrentes durante la ejecución del proyecto y optimizando los tiempos de configuración para futuros desarrolladores o mantenedores del sistema.

AffiniScore es una aplicación móvil nativa diseñada para fortalecer la relación de parejas adultas (mayores de 30 años) afectada por la rutina y la hiperconexión tecnológica. Utiliza gamificación mediante un sistema de puntajes denominado "*Affini Points*" para incentivar actos de servicio y retos de desconexión, integrando además un módulo de "**Terapeuta de Pareja IA**", comunicación privada, geofencing mediante Mapbox y un botón de emergencia S.O.S.

Este manual está específicamente orientado a la plataforma **móvil nativa**, desarrollada con **Ionic + Angular + Capacitor**, y detalla todas las herramientas, dependencias y comandos necesarios para ejecutar y compilar el sistema en entornos locales compatibles con Android.

Contar con un entorno de desarrollo correctamente configurado es esencial para garantizar la estabilidad, reproducibilidad y eficiencia del proceso de desarrollo. Además, permite una colaboración fluida entre los integrantes del equipo, facilita la detección temprana de errores y asegura que la aplicación se ejecute de manera consistente en distintos entornos de prueba.

Alcance

Este documento está enfocado exclusivamente en la **aplicación móvil nativa** del sistema **AffiniScore**, desarrollada con **Ionic + Angular + Capacitor**. Define el proceso de preparación, instalación y ejecución del entorno de desarrollo local compatible con sistemas operativos Windows 10/11 y emuladores Android.

Quedan fuera del alcance de este documento las configuraciones avanzadas de despliegue en la nube en entornos de producción (como el alojamiento en Render o la administración directa de la base de datos en Supabase), las cuales se detallarán en el repositorio.

Requisitos del Sistema

Hardware

Componente	Intel	AMD
CPU	Intel i5 o Superior	Ryzen 5 o Superior
RAM	Mínimo: 8GB	Mínimo: 8GB

Componente	Intel	AMD
	Recomendado: 16GB	Recomendado: 16GB
Espacio libre	SSD o HDD de 256GB (espacio libre mínimo 10GB)	SSD o HDD de 256GB (espacio libre mínimo 10GB)
GPU	Requerida para aceleración por hardware en emulador Android o dispositivo físico	Requerida para aceleración por hardware en emulador Android o dispositivo físico
Conectividad	Acceso estable a Internet (3 Mbps o superior)	Acceso estable a Internet (3 Mbps o superior)

Software

S.O.	Windows 10/11 (64 bits) o Ubuntu 22.04 LTS
Node.js	v20.x o superior
npm	v10.x o superior
Ionic CLI	v8.0.0 o superior
Angular CLI	v20.x o superior (Angular 20)
Capacitor CLI	v8.3.0
Java JDK	JDK 17 o superior (para compilar en Android)
Android Studio	Ladybug o superior
Emulador o Móvil	Android API 30 o superior (Android 11+)

Instalación de Herramientas

A continuación, se detallan paso a paso los procedimientos de instalación para cada herramienta requerida en el entorno de desarrollo de AffiniScore.

Instalación de Node.js y npm

Para la instalación de **Node.js** se debe primero dirigir a la página oficial nodejs.org y descargar la versión LTS más reciente (v20.x o superior). Luego de la instalación, abra la consola de Windows con **Win+R**, escriba `cmd` y presione Enter. Verifique la correcta instalación con:

```
node -v
npm -v
```

Ambos comandos deben retornar la versión instalada sin errores. Si npm no está actualizado, ejecute:

```
npm install -g npm@latest
```

Instalación de Ionic, Angular y Capacitor CLI

Para instalar y hacer uso de **Ionic**, **Angular** y **Capacitor** en el sistema, ejecute el siguiente comando en la consola. Estas herramientas permiten crear, compilar y desplegar la app móvil:

```
npm install -g @ionic/cli @angular/cli @capacitor/cli
```

Para verificar las versiones instaladas:

```
ionic --version
ng --version
cap --version
```

Instalación de Java JDK y Android Studio

Para compilar la aplicación en Android se requiere el **Java JDK 17 o superior**. Descárguelo desde adoptium.net e instale siguiendo el asistente. Luego, instale **Android Studio Ladybug o superior** desde developer.android.com/studio. Durante la instalación asegúrese de incluir:

- Android SDK (API 30 o superior)
- Android SDK Build-Tools
- Android Emulator

- Intel HAXM (para acelerar el emulador en procesadores Intel)

Una vez instalado Android Studio, configure las variables de entorno ANDROID_HOME y agregue las herramientas de platform-tools al PATH del sistema.

Instalación del Backend local (FastAPI)

Para preparar el servidor local de base de datos e inteligencia artificial, debe configurar el entorno de ejecución de Python en su estación de trabajo:

1. **Creación del Entorno Virtual:** Navegue al directorio del backend y cree un entorno virtual aislado para evitar conflictos de dependencias:

```
cd producto/app_AffiniScore/backend
python -m venv .venv
```

2. Activación del Entorno Virtual (en Windows):

```
.venv\Scripts\activate.bat
```

3. Instalación de Dependencias: Instale las librerías necesarias del backend (incluyendo FastAPI, Uvicorn, Supabase-py, y Google GenAI) a través del gestor de paquetes de Python (pip):

```
pip install -r requirements.txt
```

Ejecución del Servidor Backend

El backend actúa como puente entre la base de datos, el orquestador de Inteligencia Artificial (Gemini) y la aplicación móvil. Para iniciar el servidor de desarrollo local, ejecute el siguiente comando con el entorno virtual activo:

```
python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Clonación del Repositorio

Para poder trabajar con el repositorio desde su estación de trabajo, clone el repositorio oficial del proyecto y descargue las dependencias de Node:

```
git clone https://github.com/cristobal247/app_AffiniScore.git
cd app_AffiniScore/producto/app_AffiniScore/frontend
npm install
```

El comando `npm install` descargará automáticamente todas las dependencias declaradas en el archivo `package.json`, incluyendo Ionic, Angular, Capacitor y las librerías específicas del proyecto.

Estructura del Proyecto

La estructura base del proyecto incluye las siguientes carpetas principales dentro del directorio src/:

```
src/
├── app/
│   ├── actions/           # Lógica principal de la aplicación
│   ├── bingo/             # Acciones de negocio y lógica compartida
│   ├── forgot-password/   # Juegos de desconexión y cartones de bingo de citas
│   ├── register/         # Componentes de recuperación de contraseña
│   ├── reset-password/   # Registro de nuevos usuarios
│   ├── login/            # Reinicio de credenciales
│   ├── pages/            # Pantalla e inicio de sesión
│   ├── pipes/            # Vistas principales de la aplicación
│   ├── quick-interaction/ # Tuberías personalizadas para formateo en Angular
│   ├── services/         # Módulo de interacciones rápidas de pareja
│   ├── tabs/             # Conexión API (Supabase, FastAPI backend, Gemini)
│   ├── tienda/           # Navegación inferior principal (Tabs)
│   ├── utils/            # Gestión e intercambio de puntos y recompensas
│   ├── app.component.ts   # Funciones utilitarias y helpers
│   ├── app.routes.ts      # Componente raíz de la app
│   └── app.routes.ts      # Definición de rutas y navegación
├── assets/               # Recursos estáticos (imágenes, iconos de la app)
├── environments/         # Configuración de variables por entorno (dev/prod)
│   ├── environment.ts
│   └── environment.prod.ts
└── theme/                # Estilos globales y variables de tema (global.scss)
```

Configuración del Entorno

Variables de entorno (environment.ts)

Se debe crear o editar el archivo en `src/environments/environment.ts` para definir las credenciales de servicios externos. Este archivo **no debe ser subido al repositorio** (incluirlo en `.gitignore`):

```
export const environment = {
  production: false,
  apiUrl: 'https://affiniscare-backend.onrender.com', // URL del Backend
  FastAPI
  supabaseUrl: 'https://fojvwsegibjssttbzghe.supabase.co', // URL de Supabase
  supabaseKey: 'TU_CLAVE_PUBLICA_DE_SUPABASE',           // Clave pública
  de Supabase
  apiKeyGemini: 'TU_CLAVE_DE_GEMINI',                   // Clave de Gemini
  AI Studio
  mapboxToken: 'TU_TOKEN_DE_MAPBOX'                     // Token de Mapbox
  para mapas
};
```

Cada clave debe ser reemplazada por el valor real obtenido desde el panel de administración del servicio correspondiente (Supabase, Google AI Studio, Mapbox). Nunca exponga estas claves en repositorios públicos.

Permisos requeridos (en Android)

Para el correcto funcionamiento de las funcionalidades críticas, se requieren los siguientes permisos en el archivo `AndroidManifest.xml`:

- `android.permission.INTERNET`: Comunicación con APIs, Supabase y Gemini.
- `android.permission.CAMERA`: Escaneo de códigos QR para vinculación de parejas y subida de fotos de validación de retos.
- `android.permission.ACCESS_FINE_LOCATION` / `ACCESS_COARSE_LOCATION`: Localización para geofencing (detectar cuando la pareja está junta).
- `android.permission.ACCESS_BACKGROUND_LOCATION`: Detección en segundo plano de tiempo de calidad compartido.
- `android.permission.FOREGROUND_SERVICE` / `FOREGROUND_SERVICE_LOCATION`: Servicio persistente para el seguimiento de geofencing.
- `android.permission.POST_NOTIFICATIONS`: Alertas y recordatorios de retos pendientes.
- `android.permission.RECORD_AUDIO` / `MODIFY_AUDIO_SETTINGS`: Grabar mensaje de voz cuando se presione el botón S.O.S

Ejecución y Desarrollo

Levantar el servidor de desarrollo

Este comando inicia un servidor local y permite visualizar la aplicación en el navegador web. Ideal para pruebas rápidas de interfaz:

```
ionic serve
```

Alternativamente, el proyecto incluye un archivo automatizado desde la raíz del repositorio que configura las variables necesarias antes de levantar el servidor:

```
start_local.bat
```

Una vez iniciado, la aplicación estará disponible en el navegador en la dirección <http://localhost:4201>. Se recomienda usar Google Chrome o Brave para mejor compatibilidad con las APIs de geolocalización.

Compilación para Android

Para compilar la aplicación y ejecutarla en un dispositivo Android físico o emulador, siga estos pasos en orden:

```
ionic build  
ionic capacitor add android  
ionic capacitor open android
```

El comando **ionic build** genera los archivos estáticos de la aplicación Angular. **ionic capacitor add android** crea el proyecto nativo Android dentro de la carpeta `android/`. Finalmente, **ionic capacitor open android** abre el proyecto automáticamente en Android Studio para su compilación, depuración y emulación.

Para sincronizar cambios del código web con el proyecto nativo Android sin recrearlo desde cero, utilice:

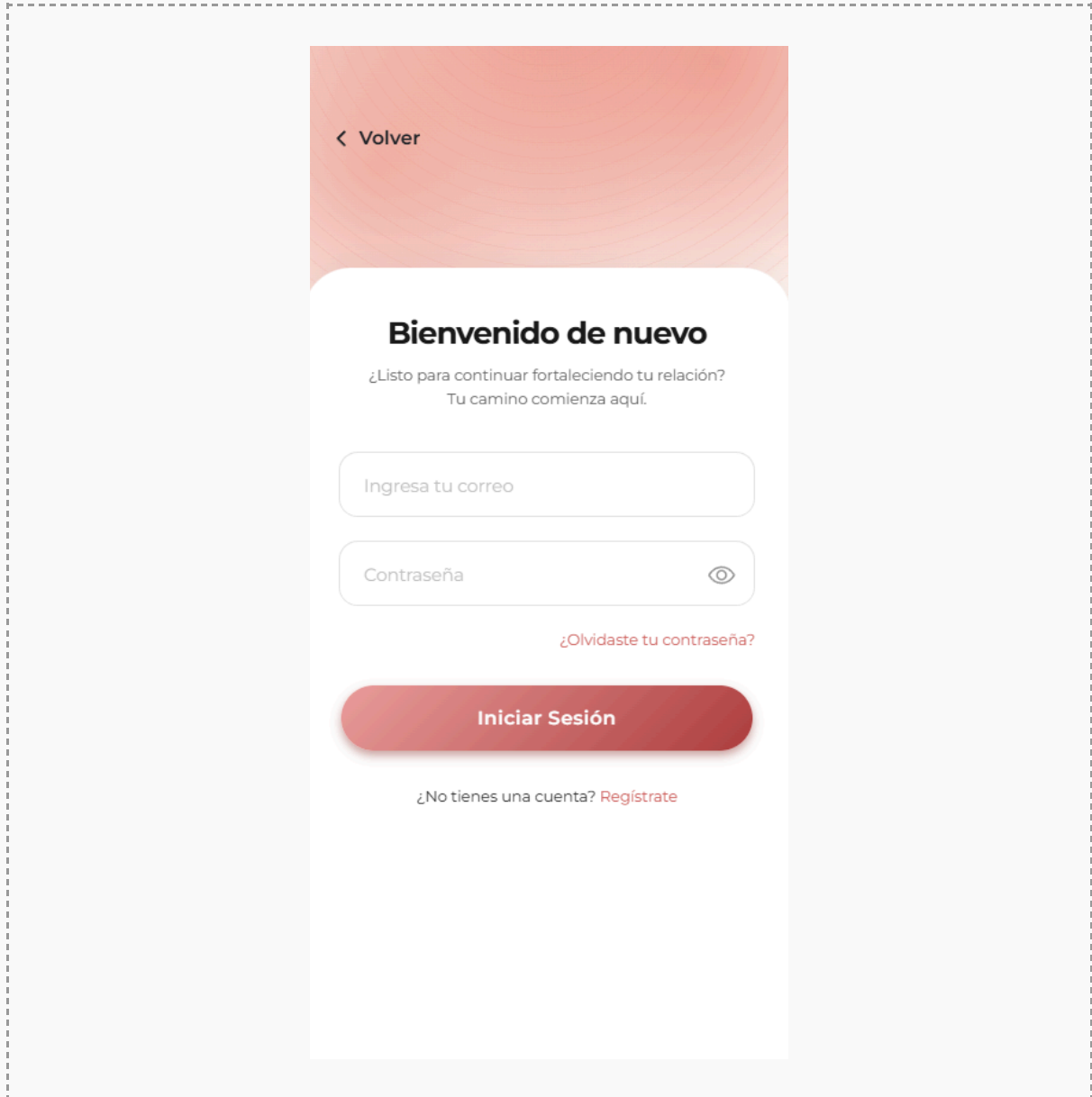
```
ionic capacitor sync android
```

Consideraciones Técnicas

- **Versión mínima de Android:** API 22 (Android 5.1)
- **Dispositivos probados:** Xiaomi Redmi, Samsung Galaxy, Emulador Pixel 5 API 30
- **Navegadores compatibles:** Google Chrome, Brave, Safari, Mozilla Firefox
- **Backend:** FastAPI hospedado en Render (<https://affiniscore-backend.onrender.com>)
- **Base de datos:** Supabase (PostgreSQL con autenticación y almacenamiento integrado)
- **IA:** Google Gemini API (para el módulo Terapeuta de Pareja IA)
- **Mapas:** Mapbox SDK (geofencing y detección de proximidad de pareja)

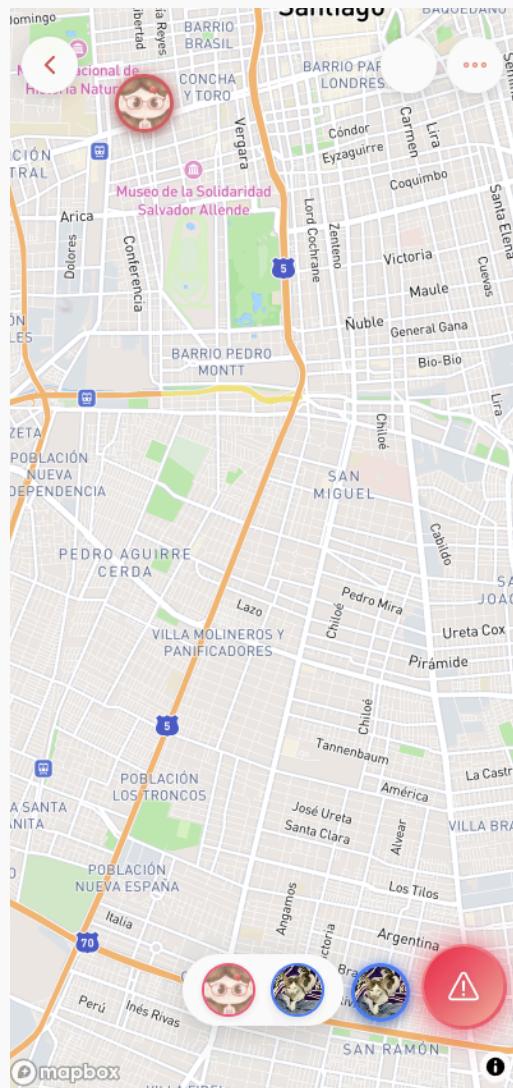
Vistas del interfaz

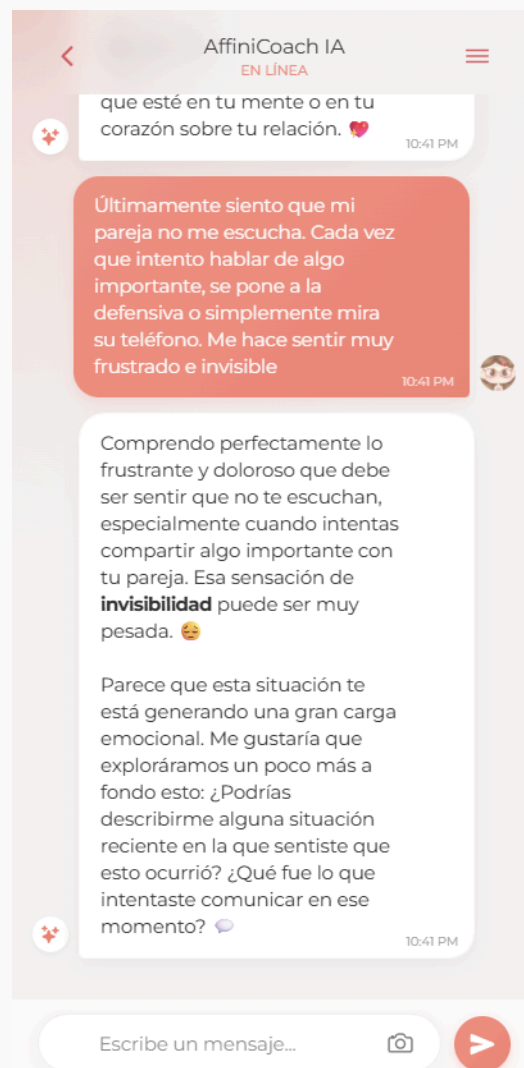
A continuación se presentan las capturas de pantalla de las principales vistas de la aplicación AffiniScore ejecutándose en el entorno de desarrollo configurado:











Conclusión

La correcta implementación de un entorno de desarrollo estructurado y documentado constituye un pilar fundamental para la escalabilidad y mantenibilidad del proyecto AffiniScore. Contar con una guía detallada como la presente permite que cualquier integrante del equipo —o un futuro mantenedor del sistema— pueda replicar el ambiente de trabajo de forma rápida y sin ambigüedades, reduciendo significativamente los tiempos de onboarding y la ocurrencia de errores de configuración.

El uso del stack híbrido *Ionic + Angular + Capacitor* ha permitido una implementación eficiente y multiplataforma, con beneficios notables en tiempo de desarrollo y compatibilidad entre dispositivos Android y navegadores web. La integración de servicios externos como Supabase, FastAPI, Google Gemini y Mapbox enriquece la experiencia de usuario al tiempo que requiere una gestión responsable de las credenciales y variables de entorno.

El equipo de desarrollo de AffiniScore se compromete a mantener este documento actualizado a medida que el proyecto evolucione, incorporando nuevas herramientas, cambios de versión o ajustes en la arquitectura del sistema. Cualquier dificultad encontrada durante la implementación puede ser abordada mediante la documentación oficial de las herramientas utilizadas y el soporte activo de la comunidad open source.